



**RAMAKRISHNA INSTITUTE OF TECHNOLOGY**

**(An Autonomous Institution)**

(Approved by AICTE, New Delhi & Permanently Affiliated to Anna University, Chennai)

Accredited by NAAC with 'A' Grade

**Pachapalayam, Perur Chettipalayam, Coimbatore-641010**



**20EEP34/ Introduction to Raspberry Pi**

## **PROJECT – I**

# **Voice Controlled Automatic Water Dispenser for using Raspberry Pi 4**

Submitted By:

Akash A – 71382303001

Amuthan A – 71382303002

Gowri B – 71382303006

Hariharasudhan M - 71382303007

Sharmili E – 71382303018

Tharun B S – 71382303020

Guided By:

**Mr. Ramkumar  
AP (Sl. Gr)/EEE**

## Introduction:

This project details the implementation of a Voice Controlled Automatic Water Dispenser system utilizing a Raspberry Pi 4. The system leverages speech recognition technology to enable hands-free dispensing of water, providing greater ease and accessibility for users. Utilizing a combination of hardware components, including sensors, relays, solenoid valves, and a microphone, alongside open-source software libraries, the system executes water dispensing commands issued via voice. The approach enhances hygiene, convenience, and accessibility, especially for individuals with mobility challenges. The project report outlines the system's design, software architecture, working principles, advantages, limitations, applications, and future prospects.

## Problem Statement:

Public water dispensers and home units commonly require manual operation, often via levers or touch switches. Such interaction is not always hygienic, especially in shared environments, and presents accessibility issues for people with physical limitations. An automated, voice-activated solution addresses these concerns by offering a hands-free, user-friendly interface for water dispensing, reducing microbial transmission risks and fostering usability for the elderly or differently-abled individuals.

## Components Required:

| Component          | Description                                   | Typical Model / Part        |
|--------------------|-----------------------------------------------|-----------------------------|
| Raspberry Pi 4     | Main controller                               | Pi 4 Model B                |
| Headphones         | Captures user voice commands and gives output | USB/3.5mm Mic               |
| Relay Module       | Isolates and switches high current            | 5V/12V Single-Channel Relay |
| Water Pump         | Pushes water through valve                    | Mini Water Pump             |
| Power Adapter      | Powers Pi and peripherals                     | 5V 3A (Pi 4 recommended)    |
| Jumper Wires       | Electrical connections                        | -                           |
| Water Tank         | Water reservoir                               | Any appropriate container   |
| Software Libraries | Speech recognition, GPIO, etc.                | Python, Speech Recognition  |

## Working Principle:

The system continuously listens for voice input through the microphone. When a predefined command (such as "dispense water," "stop water," or "pour 200ml") is recognized, the Raspberry Pi 4 processes the audio via a speech recognition module. On recognizing a valid command, it signals the relay to activate the solenoid valve, initiating water flow. The system can be programmed to dispense water for a set period (time-based) or specific volume (using a flow sensor for advanced versions). Upon completion or by receiving a stop command, the relay deactivates, stopping the water flow.

## Hardware and Software Used:

### Hardware:

- **Raspberry Pi 4 Model B:** Acts as the brain of the system.
- **Microphone:** Captures real-time voice commands.
- **Relay Module:** Electrically isolates and controls switching of the solenoid valve.
- **Solenoid Valve/Pump:** Mechanism for starting and stopping the flow of water.
- **Optional Display/Speaker:** For visual or audio feedback.

### Software:

- **Operating System:** Raspberry Pi OS (Lite or Desktop)
- **Python 3.x:** Main programming language.
- **Speech Recognition Python Library:** Handles voice processing.
- **PyAudio:** Interfaces with audio input and output devices.
- **RPi.GPIO:** Controls the GPIO pins for hardware interfacing.
- **Custom Python Scripts:** For managing command flow, error handling, and dispensing logic.

## Voice Command Flow:

1. **Idle State:** The system awaits voice input.
2. **Capture Command:** User issues a voice command (e.g., "dispense water").
3. **Speech Recognition:** Audio is processed to determine textual command.
4. **Command Validation:** Recognized command checked against a valid list.
5. **Action Trigger:**
  - If valid: GPIO outputs HIGH (activating relay → solenoid valve on).

- If invalid: System prompts user for re-entry.
6. **Dispensing State:** Water flows for set time or until “stop” command.
  7. **Completion:** System returns to idle or acknowledges completion.

### Python Code with Explanation:

```
import RPi.GPIO as GPIO
import speech_recognition as sr
import time

# GPIO pin where the relay module is connected
RELAY_PIN = 17

# Setup GPIO
GPIO.setmode(GPIO.BCM) # Use BCM pin numbering
GPIO.setup(RELAY_PIN, GPIO.OUT)
GPIO.output(RELAY_PIN, GPIO.LOW) # Ensure relay is off initially

# Initialize speech recognizer and microphone
recognizer = sr.Recognizer()
mic = sr.Microphone()

def listen_command():
    """
    Captures audio from the microphone and converts it to text using Google
    Speech Recognition API.
    Returns the recognized command as a lowercase string, or empty string on
    failure.
    """
    with mic as source:
        recognizer.adjust_for_ambient_noise(source, duration=1) # Adjust for
        ambient noise
        print("Listening for command...")
        audio = recognizer.listen(source, phrase_time_limit=5) # Listen for 5
        seconds max

    try:
        command = recognizer.recognize_google(audio).lower()
        print(f'Recognized command: {command}')
        return command
    except sr.UnknownValueError:
        print("Sorry, I could not understand the audio.")
    except sr.RequestError as e:
```

```
    print(f'Could not request results from Google Speech Recognition service;
{e}')
    return ""
```

```
def dispense_water(duration=5):
    """
```

```
    Activates the relay to open the solenoid valve and dispense water for "duration"
seconds.
    """
```

```
    print("Dispensing water...")
    GPIO.output(RELAY_PIN, GPIO.HIGH) # Turn ON relay (solenoid valve
opens)
    time.sleep(duration)
    GPIO.output(RELAY_PIN, GPIO.LOW) # Turn OFF relay (solenoid valve
closes)
    print("Stopped dispensing.")
```

```
def main():
    """
```

```
    Main loop that listens for voice commands and controls the water dispenser
accordingly.
```

```
    Recognized commands:
```

- 'dispense water' or 'start water' to begin dispensing
- 'stop' to stop dispensing immediately

```
    """
```

```
    print("Voice Controlled Water Dispenser System Started.")
    print("Say 'dispense water' to start dispensing, or 'stop' to stop.")
```

```
    dispensing = False # Flag to track dispenser state
```

```
    try:
```

```
        while True:
```

```
            command = listen_command()
```

```
            if command == "":
```

```
                continue # Skip empty recognition results
```

```
            if ('dispense water' in command or 'start water' in command) and not
dispensing:
```

```
                # Start dispensing for default 5 seconds
```

```
                dispense_water(duration=5)
```

```
                dispensing = False # since dispense_water blocks, water is off after
call
```

```

elif 'stop' in command and dispensing:
    # Immediately stop dispensing if water is flowing
    GPIO.output(RELAY_PIN, GPIO.LOW)
    dispensing = False
    print("Dispensing stopped by user command.")

else:
    print("Awaiting valid command...")

except KeyboardInterrupt:
    print("\nProgram interrupted by user.")

finally:
    GPIO.output(RELAY_PIN, GPIO.LOW) # Ensure relay off
    GPIO.cleanup()
    print("GPIO cleaned up. Program exiting.")

if __name__ == "__main__":
    main()

```

## Applications and Use Cases:

- **Hospitals and Care Centres:** Contactless access for patients with mobility limitations.
- **Public Water Dispensers:** Improved hygiene in high-traffic areas (airports, offices, parks).
- **Homes for Elderly / Disabled:** Supports independence in daily hydration.
- **Smart Kitchens:** Integration into broader home automation systems.
- **Industry:** Precise dispensing in laboratories or food & beverage production.

## Advantages & Limitations:

### Advantages:

- **Hygienic and Contactless Operation:** Reduces germ transmission.
- **Accessibility:** Benefits users with physical limitations.
- **Customizable:** Open-source flexibility enables new features.
- **Scalable:** System can expand to multi-station dispensers or integrate with IoT.

- **Cost-effective:** Utilizes readily available hardware and open-source software.

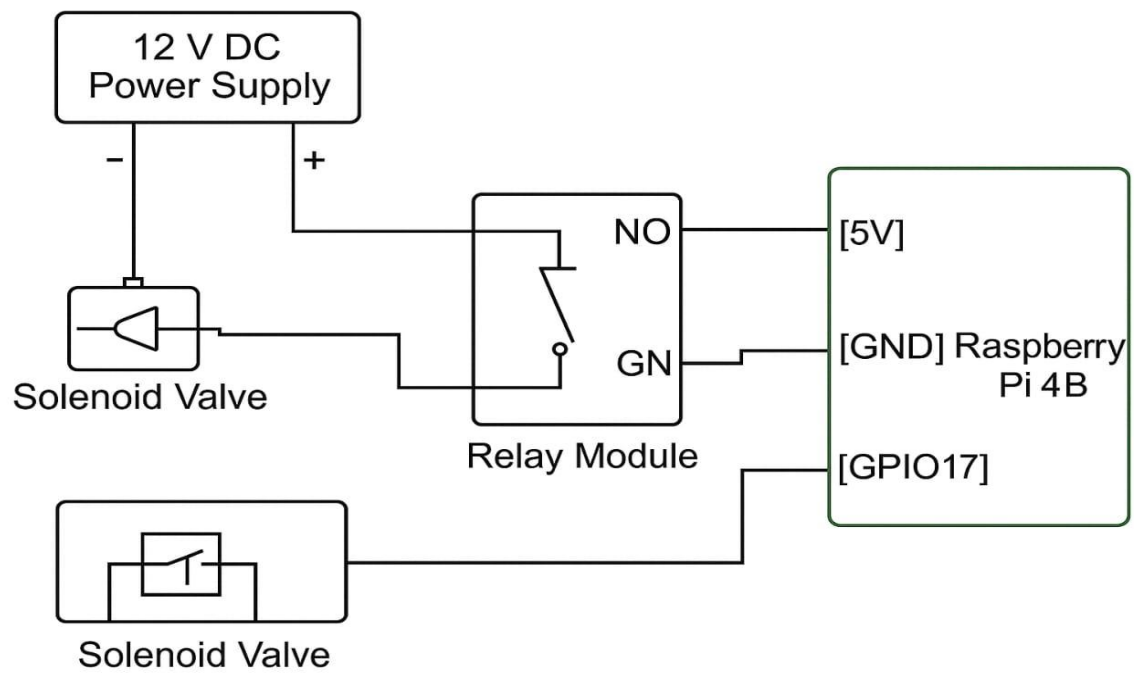
### **Limitations:**

- **Speech Recognition Accuracy:** Errors in noisy environments or with diverse accents.
- **Dependency on Power Supply:** System inoperable during outages without backup.
- **Water Quality Management:** External to the dispenser's function.
- **Initial Setup Complexity:** Requires basic knowledge of electronics and programming.
- **Limited Language Support:** Unless multi-language capability is added.

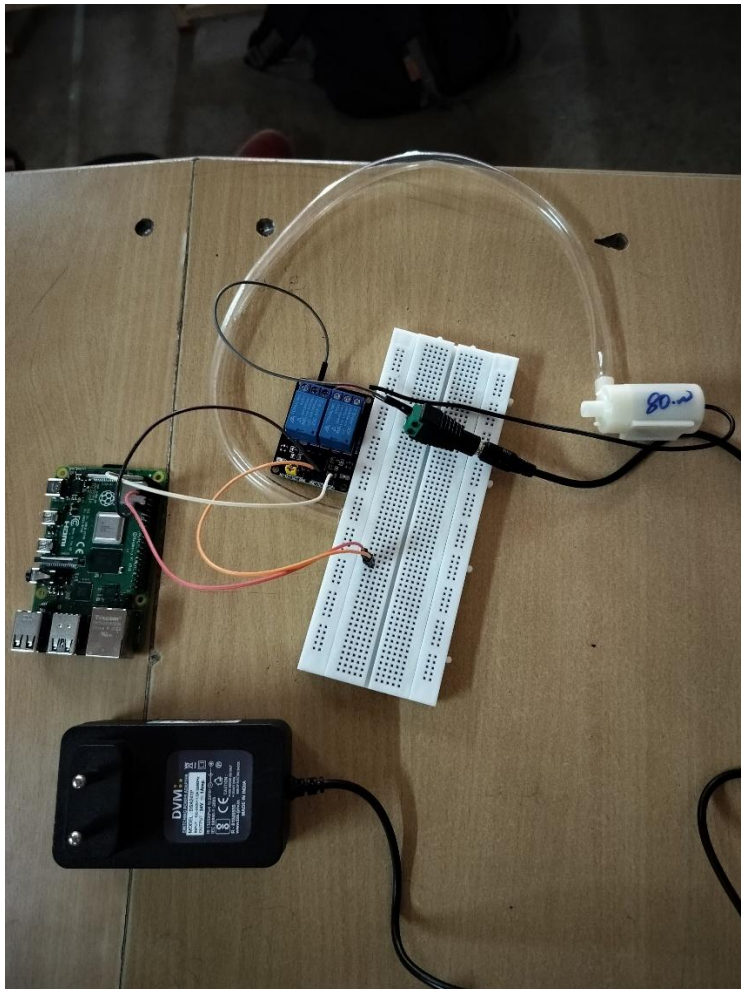
### **Future Scope:**

- **Integration with Flow Sensors:** Dispense precise water volumes by measuring real-time output.
- **Cloud Connectivity & IoT:** Monitor usage remotely, alert for maintenance or low-water status.
- **Multi-Language and Wake-Word Support:** Increase accessibility worldwide.
- **Mobile App Integration:** Allow users to configure, monitor, or trigger dispensing via mobile devices.
- **AI & Machine Learning:** Adaptive system that learns user preferences or usage patterns.
- **Energy Optimization:** Add low-power standby or solar-powered variants for rural areas.
- **Remote Voice Interface:** Use smart assistants like Alexa/Google Assistant for voice triggers.

### Connection Diagram:



### Project Prototype:





## **Conclusion:**

The **Voice Controlled Automatic Water Dispenser using Raspberry Pi 4** demonstrates a vital intersection of embedded systems, speech recognition, and automation. It addresses crucial needs for hygiene and accessibility in contemporary water dispensing, offering a modular and customizable platform for both public and private deployment. The design's open-source nature and expandability encourage future enhancements, such as precision dispensing and integration with broader IoT ecosystems. Despite minor limitations such as speech recognition challenges and setup complexity, the system presents a compelling solution that will become increasingly relevant in smart environments and assistive technology domains as these technologies advance further.